

eXtreme Programming (XP) — Conceitos, Práticas e Implementação

Sprint: Sprint 3

Data de Entrega: 22/05/2026

Foco: Conceitos centrais de cada tema e aplicação prática

1. Introdução ao eXtreme Programming (XP)

O eXtreme Programming (XP) é uma das metodologias ágeis mais consagradas para o desenvolvimento de software. Diferente de frameworks que focam majoritariamente na gestão do fluxo de trabalho, o XP eleva as boas práticas de engenharia de software a níveis "extremos". O objetivo principal é melhorar a qualidade do produto final e garantir a capacidade de adaptação da equipe frente a requisitos dinâmicos e frequentes mudanças de escopo.

2. Valores Fundamentais do XP

As decisões e práticas técnicas dentro de um ambiente XP são sustentadas por cinco valores essenciais:

- **Comunicação:** Problemas em projetos de software frequentemente derivam de falhas de comunicação. O XP prioriza a colaboração face a face, reuniões diárias (stand-ups) e o contato constante entre desenvolvedores e especialistas de negócio, minimizando a dependência de documentações extensas e obsoletas.
- **Simplicidade:** O foco está em desenvolver a funcionalidade necessária para o momento presente, sem tentar prever necessidades futuras hipotéticas (princípio YAGNI - *You Ain't Gonna Need It*). Código simples é mais fácil de testar, manter e refatorar.
- **Feedback:** Ciclos curtos de entrega e testes automatizados constantes fornecem retorno imediato sobre a qualidade do código e a aderência do produto às expectativas do cliente.
- **Coragem:** Manifesta-se na disposição para refatorar código complexo, descartar soluções que se provaram ineficazes e falar a verdade sobre prazos e viabilidade técnica.
- **Respeito:** Os membros da equipe respeitam o trabalho uns dos outros, evitam sobrecarregar os colegas com códigos mal escritos e valorizam o ritmo de trabalho sustentável.

3. Práticas Técnicas e Engenharia Ágil

- **Test-Driven Development (TDD):** Técnica onde o desenvolvimento é guiado por testes automatizados. O fluxo segue rigorosamente o ciclo *Red-Green-Refactor*: escreve-se um teste que falha, implementa-se o código mínimo para fazê-lo passar e, por fim, refatora-se a estrutura para garantir a legibilidade e boa arquitetura.
- **Pair Programming (Programação em Par):** Dois desenvolvedores dividem o mesmo contexto de trabalho. O *Piloto* foca na escrita imediata da sintaxe e lógica, enquanto o

Copiloto analisa estrategicamente a arquitetura, antecipa problemas, revisa o código em tempo real e garante a cobertura de testes.

- **Integração Contínua (CI):** Os desenvolvedores integram seus códigos ao repositório principal múltiplas vezes ao dia. Cada *commit* dispara um pipeline automatizado de compilação e execução de testes, mitigando problemas graves de conflito de código ("merge hell").

4. Exemplo Prático de Fluxo e Código (TDD)

Abaixo está representado o ciclo prático de desenvolvimento usando TDD para uma regra de negócio de cálculo de descontos em uma aplicação back-end.

Passo 1: Escrever o Teste que Falha (Red)

Antes mesmo da lógica de negócio existir, a especificação do comportamento esperado é codificada em uma suíte de testes unitários:

```
import unittest

class TestPedido(unittest.TestCase):
    def test_deve_aplicar_dez_por_cento_de_desconto_para_compras_acima_de_cem(self):
        # Arrange (Preparação)
        pedido = Pedido(valor_total=120.0)

        # Act (Ação)
        resultado = pedido.calcular_desconto()

        # Assert (Verificação)
        self.assertEqual(resultado, 12.0)

if __name__ == '__main__':
    unittest.main()
```

Nota: Este código causará um erro de compilação ou execução imediata, pois a classe Pedido ainda não foi declarada.

Passo 2: Implementação Mínima para Passar (Green)

O foco agora é puramente fazer o teste passar, escrevendo a lógica mais simples e direta possível:

```
class Pedido:
    def __init__(self, valor_total):
        self.valor_total = valor_total

    def calcular_desconto(self):
        if self.valor_total > 100.0:
            return self.valor_total * 0.1
        return 0.0
```

Nota: Ao reexecutar o teste, o resultado será verde (sucesso).

Passo 3: Refatoração (Refactor)

Com o teste garantindo o comportamento correto, o código é higienizado para adotar boas práticas de clean code, tipagem e remoção de "números mágicos":

class Pedido:

LIMITE_DESCONTO = 100.0

PERCENTUAL_DESCONTO = 0.1

def __init__(self, valor_total: float):

self.valor_total = valor_total

def calcular_desconto(self) -> float:

if self._elegivel_para_desconto():

return self.valor_total * self.PERCENTUAL_DESCONTO

return 0.0

def _elegivel_para_desconto(self) -> bool:

return self.valor_total > self.LIMITE_DESCONTO

Nota: O teste é rodado novamente para assegurar que a refatoração não quebrou a lógica preestabelecida.